

UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática



Proyecto de investigación
Maximum Clique

Integrantes: Ricardo Riveros
Gonzalo Ahumada
Daniel Muñoz
Asignatura: Optimización en Ingeniería
Profesor: Mónica Villanueva
Ayudante: Sebastián Aliaga

28 de abril de 2026

Tabla de contenidos

1. Introducción	2
2. Estado del Arte	3
3. Método	5
3.1. Formulación Matemática del Problema	5
3.1.1. Formulación como Programa Lineal Entero (ILP)	5
3.1.2. Relajación Lineal Continua (LP)	6
3.1.3. Formulación Cuadrática Continua de Motzkin-Straus	6
3.2. Procedimiento de Resolución	7
3.3. Diagrama de Flujo del Proceso	9
4. Resultados y Análisis	11
4.1. Descripción de la instancia	11
4.2. Dimensiones del modelo ILP	11
4.3. Resultados de la resolución exacta	11
4.4. Análisis de la relajación LP	12
4.5. Visualización	13
5. Discusión	14
5.1. Efectividad de la formulación ILP	14
5.2. Rol del solver HiGHS	14
5.3. Ventajas del enfoque con Pyomo	14
5.4. Comparación con enfoques alternativos	15
5.5. Limitaciones	15
6. Conclusiones	16
6.1. Trabajo futuro	16
7. Apéndice	18
7.1. Entorno de ejecución	18

7.2. Fragmento de código: modelo ILP en Pyomo	18
7.3. Instancia keller4: formato DIMACS	19
Bibliografía	20

Resumen

Este trabajo aborda el Problema del Clique Máximo (MCP), un problema clásico de optimización combinatoria clasificado como NP-duro, mediante una formulación de Programación Lineal Entera (ILP) resuelta con herramientas de modelado algebraico. Se implementa el modelo matemático en Python utilizando el framework Pyomo Bynum et al., 2021 y el solver de alto rendimiento HiGHS Huangfu y Hall, 2018, que internamente aplica el método de Ramificación y Acotamiento (Branch & Bound) con relajación LP, planos de corte y heurísticas. La metodología se evalúa sobre la instancia *keller4* del benchmark DIMACS Mascia, 2014, un grafo de 171 vértices y 9435 aristas basado en la conjetura de Keller sobre teselaciones con hipercubos. Los resultados experimentales confirman la obtención de la solución óptima $\omega = 11$ en aproximadamente 12 segundos. El análisis de la relajación LP revela una brecha de integralidad del 87.1 %, característica de la debilidad intrínseca de la formulación LP estándar para el MCP. Se concluye que la combinación de modelado algebraico declarativo con solvers MIP modernos constituye un enfoque efectivo y reproducible para la resolución exacta de problemas de optimización combinatoria.

Palabras clave: Clique Máximo, Programación Lineal Entera, Branch & Bound, Pyomo, Optimización Combinatoria, NP-duro, DIMACS.

1. Introducción

El análisis de estructuras complejas mediante la teoría de grafos ha cobrado gran relevancia en la última década debido al auge de la ciencia de datos y el análisis de redes. Dentro de este ámbito, el Problema del Clique Máximo (Maximum Clique Problem, MCP) se define formalmente como la búsqueda del subgrafo completo de mayor cardinalidad dentro de un grafo no dirigido $G = (V, E)$, donde todos los vértices del subgrafo están mutuamente conectados Pardalos y Xue, 1992.

Este trabajo tiene como objetivo principal formular, analizar y evaluar métodos algorítmicos y matemáticos —abarcando desde la optimización combinatoria exacta hasta enfoques heurísticos— para aislar y resolver de manera eficiente el MCP. La importancia de este estudio radica en la intratabilidad computacional del problema, el cual está clasificado dentro de la familia de problemas NP-duros Akbari, 2015; Szabó y Zaválnij, 2018, lo que significa que el tiempo computacional crece exponencialmente con el tamaño del grafo. A pesar de su complejidad matemática, resolver el MCP es crucial debido a su vasta aplicabilidad en áreas críticas del mundo real, tales como la bioinformática, la teoría de códigos, el reconocimiento de patrones, el diagnóstico de fallos, la visión por computadora y la economía Pardalos y Xue, 1992; Singh y Govinda, 2014. Dado que encontrar una solución exacta en grafos grandes y densos es un desafío mayor, el estudio y desarrollo de estrategias de poda, ramificación y relajación resulta fundamental. A continuación, la Sección ?? muestra el estado del arte, y la Sección ?? el método que describe la metodología propuesta, y finalmente se presentan los resultados y conclusiones.

2. Estado del Arte

La revisión de la literatura en torno al Problema del Clique Máximo (MCP) revela un esfuerzo de más de seis décadas por encontrar soluciones algorítmicas eficientes a este desafío NP-duro Akbari, 2015. A lo largo de los años, los enfoques han evolucionado desde la combinatoria discreta pura hacia técnicas de optimización continua y heurísticas avanzadas. A continuación, se detallan los avances predominantes:

1. **Complejidad y Modelos Matemáticos:** El MCP es computacionalmente equivalente al problema del Conjunto Independiente Máximo (Maximum Independent Set) y al problema de Cobertura de Vértices Mínima (Minimum Vertex Cover) Pardalos y Xue, 1992. El problema se modela comúnmente mediante programación entera (como un problema binario discreto). Alternativamente, el famoso Teorema de Motzkin-Straus demostró que el problema combinatorio discreto se puede mapear de manera exacta a un problema de programación cuadrática continua Pardalos y Xue, 1992. Desde la perspectiva de la complejidad, no solo es NP-duro encontrar la solución exacta, sino que se ha demostrado que es imposible aproximar el tamaño del clique máximo dentro de un factor constante en tiempo polinomial, a menos que $P = NP$ Pardalos y Xue, 1992.
2. **Métodos de Solución Exactos:** En el ámbito de la optimización exacta, el enfoque principal ha sido el método de Ramificación y Acotamiento (*Branch & Bound*). Históricamente, el algoritmo de Carraghan y Pardalos (1990) sentó las bases algorítmicas para los solucionadores modernos, utilizando enumeración parcial y acotamiento agresivo Szabó y Zaválnij, 2018. Solucionadores actuales de vanguardia como *cliquer* (Östergård), MCR (Tomita) y BBMC (San Segundo) son descendientes y versiones refinadas de este enfoque clásico Szabó y Zaválnij, 2018. Recientemente, se ha demostrado que reformular el problema de optimización en una serie de problemas de decisión parametrizados (búsqueda de un clique de tamaño específico k) simplifica la estructura del problema y permite reducir drásticamente el espacio de búsqueda Szabó y Zaválnij, 2018.

3. **Enfoques Heurísticos:** Dada la dificultad de los métodos exactos en grafos muy grandes, la literatura ha propuesto múltiples heurísticas para aproximar soluciones. Las más comunes son las heurísticas codiciosas secuenciales (*sequential greedy heuristics*), que construyen cliques añadiendo el mejor vértice disponible ("Best in") o eliminando el peor ("Worst out") Pardalos y Xue, 1992. Para mejorar estas aproximaciones, se aplican algoritmos de búsqueda local (como intercambios- k), búsquedas aleatorias (*Randomized algorithms*), búsqueda Tabú y redes neuronales Pardalos y Xue, 1992. Enfoques más recientes aprovechan el cálculo de Conjuntos Independientes Mínimos (Minimal Independent Sets) en cada paso recursivo, descartando tempranamente nodos que no pueden mejorar la mejor solución actual (poda), reduciendo eficientemente el árbol de exploración Singh y Govinda, 2014.
4. **Casos Polinomiales y Grafos Especiales:** Aunque el problema es NP-duro en grafos arbitrarios, el estado del arte reconoce que puede ser resuelto en tiempo polinomial en clases de grafos con topologías especiales. Destacan de manera fundamental los grafos perfectos (*perfect graphs*), grafos bipartitos, grafos de intervalos y grafos triangulados Pardalos y Xue, 1992. En estas estructuras, la relajación lineal del problema entero entrega soluciones óptimas con valores enteros de manera natural Pardalos y Xue, 1992.

3. Método

Para la realización de este estudio, la metodología se ha estructurado de la siguiente manera:

3.1. Formulación Matemática del Problema

Sea $G = (V, E)$ un grafo no dirigido con conjunto de vértices $V = \{1, 2, \dots, n\}$ y conjunto de aristas E . Se define $\bar{E} = \{(i, j) : i, j \in V, i \neq j, (i, j) \notin E\}$ como el conjunto de aristas complementarias, es decir, los pares de vértices que **no** están conectados en G .

3.1.1. Formulación como Programa Lineal Entero (ILP)

El MCP se formula como un problema de Programación Lineal Entera Binaria (Binary Integer Linear Program, BILP). Se introduce una variable de decisión binaria x_i para cada vértice $i \in V$, donde $x_i = 1$ si el vértice i pertenece al clique, y $x_i = 0$ en caso contrario Nemhauser y Wolsey, 1988; Pardalos y Xue, 1992.

Función objetivo:

$$\text{máx} \quad z = \sum_{i \in V} x_i \quad (1)$$

Restricciones de adyacencia:

$$x_i + x_j \leq 1, \quad \forall (i, j) \in \bar{E} \quad (2)$$

Restricciones de integralidad:

$$x_i \in \{0, 1\}, \quad \forall i \in V \quad (3)$$

La función objetivo (??) maximiza el número de vértices seleccionados. Las restricciones (??) garantizan que si dos vértices i y j **no** son adyacentes (es decir, $(i, j) \in \bar{E}$), entonces ambos no pueden pertenecer simultáneamente al clique. Las restricciones (??) aseguran que las variables sean binarias. Toda solución factible de este modelo corresponde a un clique en G , y la solución óptima es el clique de cardinalidad máxima Bomze et al., 1999; Pardalos y Xue, 1992.

3.1.2. Relajación Lineal Continua (LP)

Para obtener cotas superiores del valor óptimo entero, se resuelve la relajación lineal del problema (??)–(??), reemplazando la restricción de integralidad por su envolvente continua Nemhauser y Wolsey, 1988; Schrijver, 1986:

$$0 \leq x_i \leq 1, \quad \forall i \in V \quad (4)$$

Sea z_{LP}^* el valor óptimo de la relajación y z_{ILP}^* el del problema entero original. Dado que la región factible del LP contiene a la del ILP, se cumple la relación fundamental:

$$z_{ILP}^* \leq z_{LP}^* \quad (5)$$

Esta propiedad es la base teórica del método de Ramificación y Acotamiento: si la cota superior z_{LP}^* de un subproblema no supera la mejor solución entera encontrada hasta el momento, dicho subproblema puede descartarse sin pérdida de optimalidad (*poda por cota*) Nemhauser y Wolsey, 1988; Wolsey, 1998.

3.1.3. Formulación Cuadrática Continua de Motzkin-Straus

Una formulación alternativa del MCP proviene del teorema de Motzkin-Straus Motzkin y Straus, 1965, que establece una equivalencia exacta entre el problema combinatorio discreto y un problema de programación cuadrática continua. Sea A la matriz de adyacencia del grafo G y Δ el simplex estándar $\Delta = \{x \in \mathbb{R}^n : \sum_{i=1}^n x_i = 1, x_i \geq 0\}$. El teorema establece que:

$$\frac{1}{2} \left(1 - \frac{1}{\omega(G)} \right) = \max_{x \in \Delta} \frac{1}{2} x^T A x \quad (6)$$

donde $\omega(G)$ denota el tamaño del clique máximo de G . Esta formulación transforma el problema discreto en uno de optimización continua no lineal, lo cual permite emplear técnicas de programación cuadrática para su resolución Bomze et al., 1999; Pardalos y Xue, 1992.

3.2. Procedimiento de Resolución

1. **Herramientas de desarrollo:** Para los experimentos, el equipo computacional ejecutará algoritmos implementados en el lenguaje de programación Python. Python ofrece una robusta capacidad para la formulación rápida de modelos de optimización y el manejo eficiente de matrices de adyacencia.
2. **Materiales de prueba:** Los materiales consistirán en grafos estándar extraídos de bancos de pruebas (*benchmarks*) ampliamente reconocidos en la literatura para evaluar la búsqueda exacta de cliques, tales como los grafos del *2nd DIMACS Challenge* o las instancias de la librería BHOSLIB Johnson y Trick, 1993; Pardalos y Rodgers, 1992; Xu, 2000.
3. **Procedimiento algorítmico central:** El procedimiento se basará en el método exacto de Ramificación y Acotamiento (*Branch & Bound*) Carraghan y Pardalos, 1990; Pardalos y Rodgers, 1992.
4. **Cálculo de la relajación:** El algoritmo comenzará en cada paso resolviendo la relajación lineal o continua del problema original de programación entera Nemhauser y Wolsey, 1988.
5. **Proceso de ramificación:** Si la solución óptima del problema relajado proporciona un vértice con valores fraccionarios, el procedimiento continuará ramificando sobre una variable discreta x_k que no posea un valor entero Carraghan y Pardalos, 1990; Nemhauser y Wolsey, 1988. Dado que el problema se modela con variables binarias ($x_j \in \{0, 1\}$), en cada ramificación se fijará el valor de la variable creando dos subproblemas mutuamente excluyentes: $x_k = 0$ y $x_k = 1$

Algorithm 1 Ramificación y Acotamiento (Branch & Bound) para el MCP

Require: Grafo $G = (V, E)$, Formulación binaria del problema (Maximización)

Ensure: Tamaño del clique máximo $\tilde{z}$ y el vector de solución x^*

```
1:  $L \leftarrow \{P_0\}$  ▷ Inicializar la lista de nodos con el problema original
2:  $\tilde{z} \leftarrow 0$  ▷ Mejor cota inferior encontrada (tamaño máximo del clique)
3:  $x^* \leftarrow \emptyset$  ▷ Vector solución óptimo
4: while  $L \neq \emptyset$  do
5:   Seleccionar y remover un subproblema  $P$  de la lista  $L$ 
6:   Resolver la relajación lineal de  $P$  para obtener la solución  $x'$  y su valor  $z'$ 
7:   if  $P$  es infactible then
8:     continue ▷ Poda por infactibilidad
9:   end if
10:  if  $z' \leq \tilde{z}$  then
11:    continue ▷ Poda por cota: el nodo no puede mejorar la mejor solución
12:  end if
13:  if  $x'$  es completamente entera then
14:     $\tilde{z} \leftarrow z'$  ▷ Actualizar la mejor solución encontrada
15:     $x^* \leftarrow x'$ 
16:  else
17:    Seleccionar una variable fraccionaria  $x_k$  del vector  $x'$ 
18:    Crear subproblema  $P_0$  añadiendo la restricción  $x_k = 0$  a  $P$ 
19:    Crear subproblema  $P_1$  añadiendo la restricción  $x_k = 1$  a  $P$ 
20:     $L \leftarrow L \cup \{P_0, P_1\}$  ▷ Ramificar el espacio de búsqueda
21:  end if
22: end while
23: return  $\tilde{z}, x^*$ 
```

3.3. Diagrama de Flujo del Proceso

La Figura ?? presenta el diagrama de flujo del procedimiento implementado en el notebook experimental. Se utiliza Pyomo como framework de modelado algebraico y HiGHS como solver MIP, el cual resuelve internamente mediante Branch & Bound con relajación LP, cortes y heurísticas.

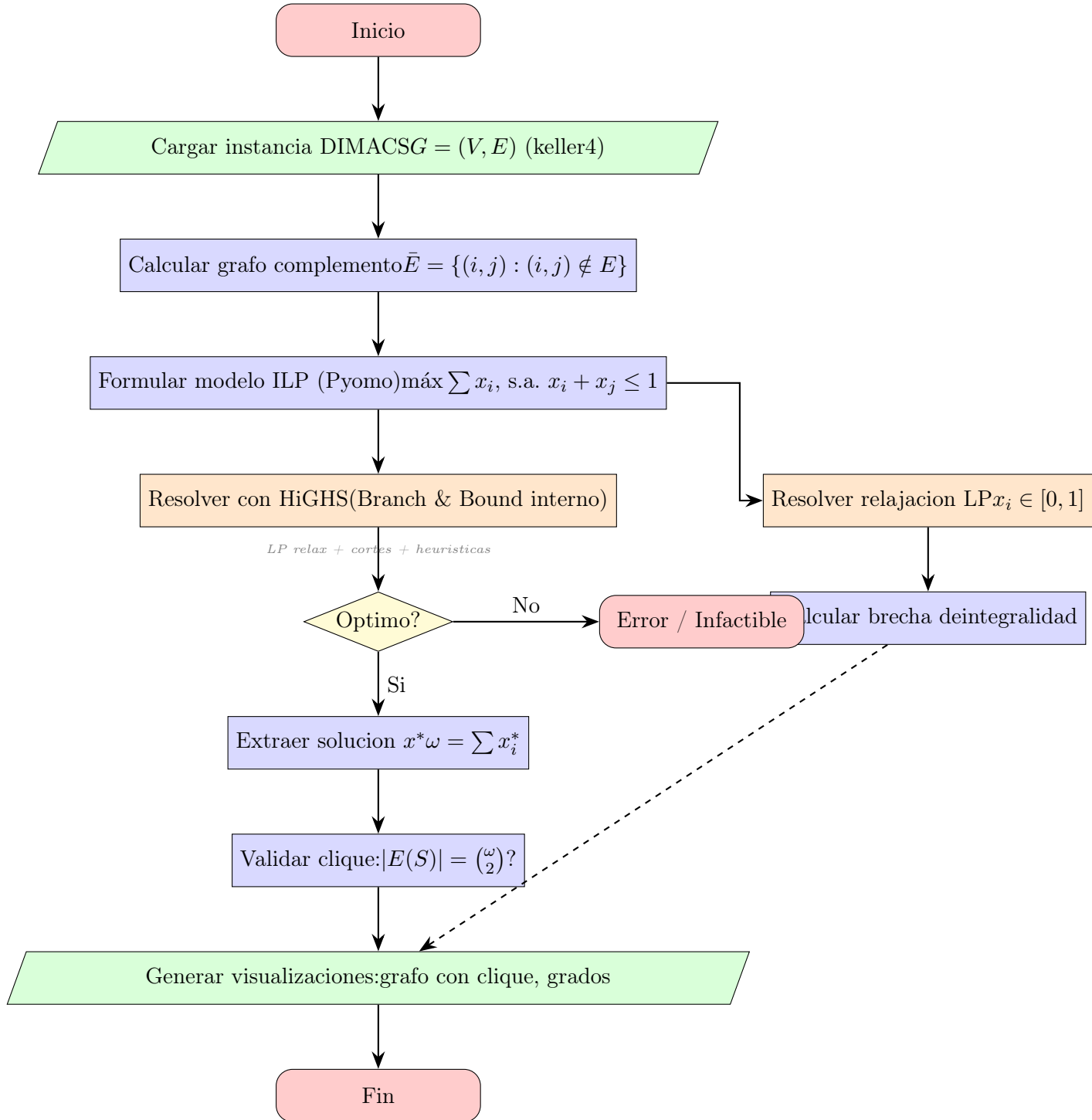


Figura 1: Diagrama de flujo de la implementación con Pyomo y HiGHS para el MCP.

4. Resultados y Análisis

En esta sección se presentan los resultados obtenidos al resolver el Problema del Clique Máximo sobre la instancia *keller4* del benchmark DIMACS Mascia, 2014, utilizando la formulación ILP descrita en la Sección ?? implementada en Pyomo Bynum et al., 2021 con el solver HiGHS Huangfu y Hall, 2018.

4.1. Descripción de la instancia

La instancia *keller4* pertenece a la familia Keller, basada en la conjetura de Keller sobre teselaciones del espacio mediante hipercubos Mascia, 2014. Sus características se resumen en la Tabla ??.

Tabla 1: Características de la instancia *keller4*.

Propiedad	Valor
Vértices ($ V $)	171
Aristas ($ E $)	9 435
Densidad	0.6491
Clique máximo conocido (ω)	11
Aristas del complemento ($ \bar{E} $)	5 100

4.2. Dimensiones del modelo ILP

El modelo formulado en Pyomo contiene 171 variables binarias de decisión (una por vértice) y 5 100 restricciones de no-adyacencia, correspondientes a las aristas del grafo complemento $\bar{G}$. La ratio de restricciones por variable es de 29.8, lo que indica un modelo fuertemente restringido.

4.3. Resultados de la resolución exacta

El solver HiGHS resolvió el modelo ILP hasta optimalidad, encontrando un clique de cardinalidad $\omega = 11$, lo cual coincide con el óptimo conocido de la instancia.

Los resultados se resumen en la Tabla ??.

Tabla 2: Resultados de la resolución del MCP en keller4.

Métrica	Valor
Clique máximo encontrado (ω)	11
Óptimo conocido	11
Tiempo de resolución	≈ 12 s
Estado del solver	Óptimo
Solver	HiGHS
Framework	Pyomo

La solución fue validada verificando que el subgrafo inducido por los 11 vértices del clique es completo, es decir, contiene exactamente $\binom{11}{2} = 55$ aristas, todas presentes en el grafo original.

4.4. Análisis de la relajación LP

Para evaluar la calidad de la formulación, se resolvió la relajación lineal continua del modelo (reemplazando $x_i \in \{0, 1\}$ por $x_i \in [0, 1]$). Los resultados se presentan en la Tabla ??.

Tabla 3: Comparación entre solución entera y relajación LP.

Métrica	Valor
Cota LP (relajación)	85.50
Solución entera (ILP)	11
Brecha de integralidad	87.1 %

La brecha de integralidad de 87.1 % indica que la relajación LP proporciona una cota superior muy débil para esta instancia. Este fenómeno es bien conocido en la literatura: la formulación estándar del MCP basada en restricciones de pares no adyacentes produce relajaciones LP intrínsecamente débiles, especialmente en grafos con estructura regular Bomze et al., 1999; Nemhauser y Wolsey, 1988. Esto implica que

el solver debe explorar un árbol de Branch & Bound extenso para cerrar la brecha entre la cota LP y la solución entera óptima.

4.5. Visualización

La Figura ?? muestra el grafo keller4 con el clique máximo destacado en rojo. Los vértices pertenecientes al clique se identifican con mayor tamaño y las aristas entre ellos se resaltan. La distribución de grados del grafo muestra que los vértices del clique poseen grados superiores al promedio, lo cual es consistente con la estructura esperada de un clique en un grafo denso.

5. Discusión

5.1. Efectividad de la formulación ILP

La formulación ILP estándar del MCP, basada en restricciones sobre pares no adyacentes (Ecuaciones (??)–(??)), demostró ser suficiente para resolver la instancia keller4 hasta optimalidad en un tiempo computacional razonable (≈ 12 s). Esto valida el enfoque de resolución exacta mediante solvers MIP modernos para instancias de tamaño moderado Wolsey, 1998.

Sin embargo, la elevada brecha de integralidad (87.1 %) revela una limitación estructural de esta formulación. La relajación LP no captura adecuadamente la combinatoria del problema, lo que obliga al solver a realizar un esfuerzo significativo de ramificación. En la literatura se han propuesto formulaciones reforzadas mediante desigualdades de clique, restricciones de rango y coloreo fraccionario que producen relajaciones más ajustadas Bomze et al., 1999; Schrijver, 1986.

5.2. Rol del solver HiGHS

HiGHS Huangfu y Hall, 2018 demostró ser significativamente más eficiente que GLPK para esta clase de problemas. Mientras GLPK no logró resolver la instancia en tiempos razonables (más de 5 minutos sin convergencia), HiGHS alcanzó la optimalidad en aproximadamente 12 segundos. Esta diferencia se atribuye a las técnicas avanzadas implementadas en HiGHS: preprocesamiento, generación de cortes (Gomory, MIR), heurísticas de factibilidad y estrategias de ramificación adaptativas.

5.3. Ventajas del enfoque con Pyomo

El uso de Pyomo como framework de modelado algebraico ofrece ventajas metodológicas relevantes Bynum et al., 2021:

- **Separación modelo-solver:** La formulación matemática es independiente del solver utilizado. Cambiar de HiGHS a CPLEX o Gurobi requiere modificar una sola línea de código.

- **Legibilidad:** El modelo en Pyomo refleja directamente la formulación matemática, facilitando la verificación y reproducibilidad.
- **Extensibilidad:** Es directo agregar restricciones adicionales (simetría, cortes de usuario) sin reestructurar el código.

5.4. Comparación con enfoques alternativos

Se consideraron dos enfoques complementarios durante el desarrollo:

1. **Branch & Bound manual con PyBnB** Hackebeit, 2020: Implementación explícita del algoritmo B&B usando `scipy` Virtanen et al., 2020 para resolver las relajaciones LP en cada nodo. Este enfoque ofrece control total sobre la estrategia de ramificación y poda, pero requiere mayor complejidad de implementación.
2. **Pyomo + solver MIP:** El solver gestiona internamente todo el proceso de B&B. Se pierde visibilidad sobre el árbol de búsqueda, pero se gana eficiencia y robustez.

Para los objetivos del presente trabajo —demostrar la aplicación de optimización exacta al MCP—, el enfoque con Pyomo resulta más adecuado por su claridad y concisión, delegando la complejidad algorítmica al solver.

5.5. Limitaciones

Las principales limitaciones identificadas son:

- La formulación ILP estándar no escala bien a instancias de gran tamaño ($|V| > 500$) debido al número cuadrático de restricciones ($O(|\bar{E}|)$).
- La brecha de integralidad elevada sugiere que formulaciones alternativas o técnicas de corte serían necesarias para instancias más difíciles.
- El análisis se limita a una instancia del benchmark DIMACS; un estudio más completo requeriría evaluar múltiples familias de grafos.

6. Conclusiones

A partir del trabajo realizado, se establecen las siguientes conclusiones:

1. **Formulación exitosa:** El Problema del Clique Máximo fue formulado correctamente como un Programa Lineal Entero Binario y resuelto hasta optimalidad mediante el solver HiGHS, obteniendo $\omega = 11$ para la instancia keller4, coincidiendo con el óptimo conocido del benchmark DIMACS.
2. **Validez del enfoque de modelado algebraico:** Pyomo demostró ser una herramienta efectiva para la implementación de modelos de optimización combinatoria, permitiendo una traducción directa de la formulación matemática a código ejecutable sin sacrificar legibilidad ni generalidad.
3. **Importancia del solver:** La elección del solver tiene un impacto determinante en la viabilidad práctica. HiGHS resolvió en ≈ 12 segundos lo que GLPK no logró en más de 5 minutos, evidenciando que las técnicas modernas de resolución MIP (cortes, heurísticas, preprocesamiento) son esenciales para abordar problemas NP-duros de tamaño real.
4. **Debilidad de la relajación LP:** La brecha de integralidad de 87.1 % confirma que la formulación estándar basada en restricciones de pares produce relajaciones LP débiles para el MCP. Esto es consistente con los resultados teóricos de la literatura Bomze et al., 1999 y motiva el uso de desigualdades válidas adicionales en trabajos futuros.
5. **Reproducibilidad:** La implementación completa en un Jupyter Notebook Kluyver et al., 2016 con dependencias open-source (Python Python Software Foundation, 2024, Pyomo, HiGHS, NetworkX Hagberg et al., 2008) garantiza la reproducibilidad de los resultados.

6.1. Trabajo futuro

Se identifican las siguientes líneas de extensión:

- Evaluar formulaciones reforzadas con desigualdades de clique y cortes de odd-hole.
- Comparar el rendimiento sobre múltiples familias del benchmark DIMACS (C, brock, p_hat, gen).
- Implementar un enfoque híbrido que combine heurísticas (greedy, búsqueda local) con la verificación exacta de optimalidad.
- Explorar la formulación cuadrática de Motzkin-Straus Motzkin y Straus, 1965 como alternativa de resolución.

7. Apéndice

7.1. Entorno de ejecución

La Tabla ?? detalla las herramientas y versiones utilizadas en la implementación experimental.

Tabla 4: Herramientas de software utilizadas.

Herramienta	Versión	Propósito
Python Python Software Foundation, 2024	3.12+	Lenguaje de programación
Pyomo Bynum et al., 2021	6.7+	Modelado algebraico de optimización
HiGHS Huangfu y Hall, 2018	1.7+	Solver LP/MIP
NetworkX Hagberg et al., 2008	3.2+	Manipulación de grafos
NumPy Harris et al., 2020	1.26+	Cómputo numérico
Matplotlib Hunter, 2007	3.8+	Visualización
Jupyter Kluyver et al., 2016	7.0+	Notebook interactivo

7.2. Fragmento de código: modelo ILP en Pyomo

A continuación se presenta el código principal de la formulación del MCP en Pyomo:

```
import pyomo.environ as pyo

def build_mcp_model(G):
    model = pyo.ConcreteModel("MaximumClique")
    model.V = pyo.Set(initialize=list(G.nodes()))
    model.x = pyo.Var(model.V, domain=pyo.Binary)
    model.obj = pyo.Objective(
        expr=sum(model.x[i] for i in model.V),
        sense=pyo.maximize
    )
```

```

model.nonadj = pyo.ConstraintList()
for i in G.nodes():
    for j in G.nodes():
        if i < j and not G.has_edge(i, j):
            model.nonadj.add(
                model.x[i] + model.x[j] <= 1
            )
return model

model = build_mcp_model(G)
solver = pyo.SolverFactory('appsi_highs')
result = solver.solve(model)

```

7.3. Instancia keller4: formato DIMACS

La instancia se distribuye en formato ASCII estándar DIMACS con la siguiente estructura:

```

c keller4.clq
c Keller graph on 4 dimensions
p col 171 9435
e 1 2
e 1 3
...

```

Donde `p col 171 9435` define un grafo de 171 vértices y 9435 aristas, y cada línea `e u v` define una arista entre los vértices u y v . El archivo completo está disponible en el repositorio del proyecto.

Referencias

- Akbari, Z. O. (2015). A Polynomial-Time Algorithm for the Maximum Clique Problem. *Department of Mathematics and Computer Science, Friedrich Schiller University Jena.*
- Bomze, I. M., Budinich, M., Pardalos, P. M., & Pelillo, M. (1999). The maximum clique problem. *Handbook of Combinatorial Optimization*, 4, 1-74.
- Bynum, M. L., Hackebeit, G. A., Hart, W. E., Laird, C. D., Nicholson, B. L., Sirola, J. D., Watson, J.-P., & Woodruff, D. L. (2021). Pyomo—optimization modeling in Python (3.^a ed.). *Springer Optimization and Its Applications*, 67. <https://doi.org/10.1007/978-3-030-68928-5>
- Carraghan, R., & Pardalos, P. M. (1990). An exact algorithm for the maximum clique problem. *Operations Research Letters*, 9(6), 375-382.
- Hackebeit, G. A. (2020). PyBnB: A parallel branch-and-bound engine for Python [MIT License].
- Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference (SciPy2008)*, 11-15.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Huangfu, Q., & Hall, J. A. J. (2018). HiGHS—A high performance software for linear optimization. *Mathematical Programming Computation*, 10(4), 557-592. <https://doi.org/10.1007/s12532-017-0130-5>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
- Johnson, D. S., & Trick, M. A. (1993). The Second DIMACS Implementation Challenge: NP Hard Problems: Maximum Clique, Graph Coloring, and Satisfiability.
- Kluyver, T., Ragan-Kelley, B., Pérez, F., et al. (2016). Jupyter Notebooks—a publishing format for reproducible computational workflows.

- Mascia, F. (2014). DIMACS Benchmark Set for the Maximum Clique Problem [IRIDIA, Université Libre de Bruxelles. Consultado en abril 2026].
- Motzkin, T. S., & Straus, E. G. (1965). Maxima for graphs and a new proof of a theorem of Turán. *Canadian Journal of Mathematics*, 17, 533-540. <https://doi.org/10.4153/CJM-1965-053-6>
- Nemhauser, G. L., & Wolsey, L. A. (1988). *Integer and Combinatorial Optimization*. Wiley, New York.
- Pardalos, P. M., & Rodgers, G. P. (1992). A branch and bound algorithm for the maximum clique problem. *Computers and Operations Research*, 19(5), 363-375.
- Pardalos, P. M., & Xue, J. (1992). *The Maximum Clique Problem* (inf. téc.). Department of Industrial y Systems Engineering, University of Florida.
- Python Software Foundation. (2024). Python Language Reference, version 3.
- Schrijver, A. (1986). *Theory of Linear and Integer Programming*. John Wiley & Sons, Chichester.
- Singh, K. K., & Govinda, L. (2014). A Simple and Efficient Heuristic Algorithm For Maximum Clique Problem. *2014 IEEE 8th International Conference on Intelligent Systems and Control (ISCO)*, 269-273.
- Szabó, S., & Zaválnij, B. (2018). A Different Approach to Maximum Clique Search. *2018 20th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*, 310-316.
- Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261-272. <https://doi.org/10.1038/s41592-019-0686-2>
- Wolsey, L. A. (1998). *Integer Programming*. John Wiley & Sons, New York.
- Xu, K. (2000). BHOSLIB: Benchmarks with Hidden Optimum Solutions for Graph Problems.